Történelem Vázlat

Fejlesztői dokumentáció

Siófoki SZC Mathiász János Technikum, Gimnázium és Kollégium

Szoftverfejlesztő és – tesztelő képzés

Konzulens:

Holczman József

Készítette:

Szabó Máté, Zsupos Dominik

Készítés időszaka:

2025. november 11. – 2026. április 20.

Rendszer architektúrája

A webalkalmazás háromrétegű (3-tier) architektúrára épül, amely biztosítja a rendszer modularitását, skálázhatóságát és könnyű karbantarthatóságát.

Frontend (kliens oldal)

- React alapú Single Page Application (SPA)
- Feladata:
    - felhasználói felület megjelenítése
    - felhasználói interakciók kezelése
    - API hívások indítása a backend felé

Backend (szerver oldal)

- Node.js + Express alapú REST API
- Feladata:
    - üzleti logika kezelése
    - adatfeldolgozás
    - hitelesítés és jogosultságkezelés (JWT)

Adatbázis réteg

- MySQL relációs adatbázis
- Feladata:
    - adatok tartós tárolása
    - relációk kezelése
    - adatintegritás biztosítása

Adatáramlás folyamata

1. A felhasználó interakcióba lép a frontend felülettel
2. A frontend HTTP kérést küld a backend API-nak
3. A backend feldolgozza a kérést, szükség esetén adatbázis lekérdezést végez
4. A backend JSON formátumban visszaküldi az eredményt
5. A frontend megjeleníti az adatokat

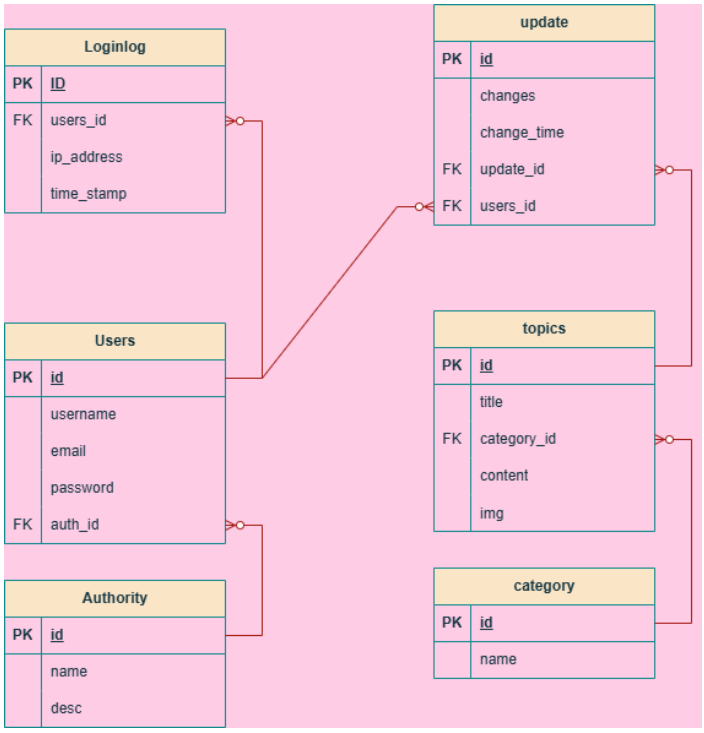
Ez az architektúra biztosítja a rendszer átláthatóságát és lehetővé teszi a jövőbeni bővítést (pl. mobilalkalmazás integráció).

Funkciók bemutatása

Adatbázis

Az alkalmazás működéséhez szükséges adatok tárolása egy MySQL alapú relációs adatbázisban történik. A relációs adatmodell lehetővé teszi az információk strukturált, jól áttekinthető és hatékony kezelését, miközben biztosítja az egyes adategységek közötti kapcsolatok egyértelmű meghatározását.

Az adatbázis tervezése során kiemelt szempont volt az átláthatóság, a bővíthetőség, valamint az adatintegritás fenntartása. Ennek megfelelően a rendszer több, egymással kapcsolatban álló táblából épül fel, amelyek különböző funkciókat látnak el.



1.. ábra: A végleges adatbázis terv ER diagramja 2025. 11. 28.

Alkalmazott táblák

Az adatbázis az alábbi főbb táblákat tartalmazza:

- **users** – a felhasználók adatait tárolja (név, email, jelszó)
- **authority** – a felhasználói jogosultsági szinteket kezeli
- **topics** – a történelmi témák alapadatait tartalmazza
- **category** – a témák kategorizálásáért felel

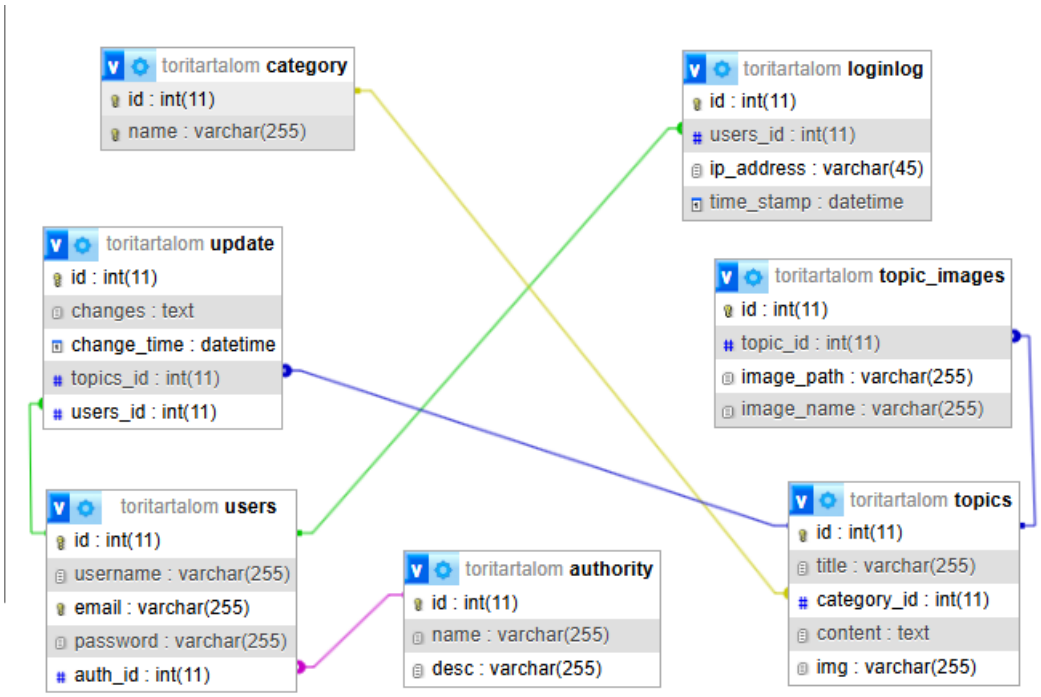
- **topic_images** – a témákhoz tartozó képek tárolására szolgál
- **update** – a tartalmi módosítások nyomon követését biztosítja
- **loginlog** – a bejelentkezési eseményeket rögzíti

Táblák közötti kapcsolatok

A táblák között logikai kapcsolatok kerültek kialakításra annak érdekében, hogy az adatok egymással összekapcsolhatók és konzisztensen kezelhetők legyenek. Például:

- Egy felhasználó egy adott jogosultsági szinthez tartozik (**users – authority**)
- Egy téma egy kategóriába sorolható (**topics – category**)
- Egy témához több kép is tartozhat (**topics – topic_images**)

A kapcsolatok idegen kulcsok (foreign key) segítségével valósulnak meg, amelyek biztosítják a hivatkozások érvényességét és az adatok konzisztenciáját.



2. ábra: A végleges adatbázis ER diagramja

Kulcsok és adatintegritás

Minden tábla rendelkezik elsődleges kulccsal (primary key), amely egyértelműen azonosítja az egyes rekordokat. Az idegen kulcsok alkalmazása megakadályozza az érvénytelen kapcsolatok létrejöttét az adatbázisban.

A tervezés során fontos cél volt a redundancia minimalizálása, amelyet a normalizálás alapelveinek alkalmazásával sikerült elérni. Ennek eredményeként az adatok nem ismétlődnek feleslegesen, ami hatékonyabb tárolást és egyszerűbb karbantartást tesz lehetővé.

Adatbázis-struktúra

A rendszer három fő táblára épül:

- **users**
  - id, username, email, password, auth_id
  - *(auth_id: 1 = admin, 2 = felhasználó)*
- **topics**
  - id, title, content, category_id, img
- **categories**
  - id, name

Ez a struktúra támogatja a tartalmak hierarchikus kezelését, valamint lehetővé teszi a dinamikus frontend megjelenítést.

Frontend komponensek

A frontend rendszer React alapú, komponens-alapú felépítésű, amely moduláris szerkezetével lehetővé teszi a könnyű karbantartást, új funkciók integrálását és a felhasználói élmény optimalizálását. A webalkalmazás minden vizuális elemét és interaktív funkcióját önálló komponensek kezelik, amelyek együttműködve biztosítják a dinamikus és reszponzív működést.

A komponensek célja:

- a történelmi tartalmak strukturált és áttekinthető megjelenítése,
- a felhasználói navigáció egyszerűsítése és logikus szervezése,
- az adminisztrációs és szerkesztői funkciók kezelése,
- valamint a modern UI/UX elvek alkalmazása a felhasználói élmény növelése érdekében.

AdminDashboard komponens

Az AdminDashboard az admin felület fő kiindulópontja, amely kártyákon keresztül biztosít gyors hozzáférést a rendszer fő funkcióihoz (felhasználók, témák kezelése). Ez a felépítés azért

előnyös, mert a vizuálisan szeparált kártyák segítik a gyors döntéshozatalt és csökkentik a navigációs hibákat.

Fő jellemzők:

- Link alapú SPA navigáció (oldalváltás újratöltés nélkül)
- Reszponzív Bootstrap grid (különböző kijelzőkhöz igazodik)
- Kártyás UI, ami jól elkülöníti a funkciókat
- Egyértelmű struktúra: cím – leírás – művelet

AdminRoute komponens

Az **AdminRoute komponens** a rendszer védett útvonalainak kezeléséért felel, és biztosítja, hogy az adminisztrátori felületek csak megfelelő jogosultsággal legyenek elérhetők. Amennyiben a felhasználó nem rendelkezik admin jogokkal, a rendszer automatikusan visszairányítja a főoldalra, ezzel megakadályozva a jogosulatlan hozzáférést.

Főbb jellemzők:

- **Jogosultság ellenőrzés:** Az AuthContext segítségével vizsgálja, hogy a felhasználó admin szerepkörrel rendelkezik-e (isAdmin() függvény).
- **Feltételes renderelés:** Csak abban az esetben jeleníti meg a gyermek komponenseket, ha a jogosultság megfelelő, ellenkező esetben átirányítást hajt végre.
- **Wrapper működés:** A komponens burkoló elemként funkcionál, amely a hozzá tartozó route-ok elérését szabályozza.

Technológiai részletek:

- **AuthContext:** Globális szinten biztosítja a felhasználói állapot és jogosultságok elérését.
- **React Router:** A Navigate komponens végzi az átirányítást, fenntartva az SPA működést.
- **React:** Funkcionális komponensként van megvalósítva, így egyszerűen újrahasználható és könnyen integrálható.

AllTopic komponens

Az **AllTopic komponens** feladata, hogy az alkalmazásban elérhető összes történelmi témát átlátható és vizuálisan rendezett formában jelenítse meg az adminisztrátorok számára. Lehetőséget biztosít a tananyagok áttekintésére, valamint új témák létrehozására és a meglévők kezelésére. A komponens a frontend SPA architektúra részeként működik, és közvetlenül kommunikál a backend REST API-val az adatok lekéréséhez.

Fő funkciók:

- Backendből történő adatlekérés
- Kártyás megjelenítés
- „Megnyitás" navigáció
- Visszalépés lehetőség

AuthContext komponens

Az AuthContext a frontend alkalmazás központi autentikációs és jogosultságkezelő rétege. Feladata a felhasználói állapot globális kezelése, valamint a bejelentkezési és jogosultsági logika egységes biztosítása.

Főbb funkciók:

- A bejelentkezett felhasználó adatainak tárolása (JWT alapján)
- Token ellenőrzése oldalbetöltéskor
- Be- és kijelentkezési műveletek kezelése
- Admin jogosultság ellenőrzése (isAdmin)
- Globális elérhetőség biztosítása Context API segítségével

Technológiai háttér:

- React Context API és hookok (useState, useEffect)
- JWT token dekódolás
- Jogosultságkezelés frontend oldalon (backend ellenőrzéssel)

CreateTopic komponens

A CreateTopic komponens az adminisztrációs felületen új történelmi témák létrehozását teszi lehetővé egy vizuális szerkesztővel támogatott űrlapon keresztül.

Főbb funkciók:

- Új téma létrehozása (cím, tartalom, kép)
- WYSIWYG szerkesztés (TinyMCE)
- Kép Base64 formátumba alakítása
- Adatok elküldése backend API-n keresztül
- Kategória azonosítása URL paraméterből

Technológiai háttér:

- React hookok és routing (useState, useParams)
- TinyMCE editor integráció
- REST API kommunikáció JWT hitelesítéssel

Főoldal komponens

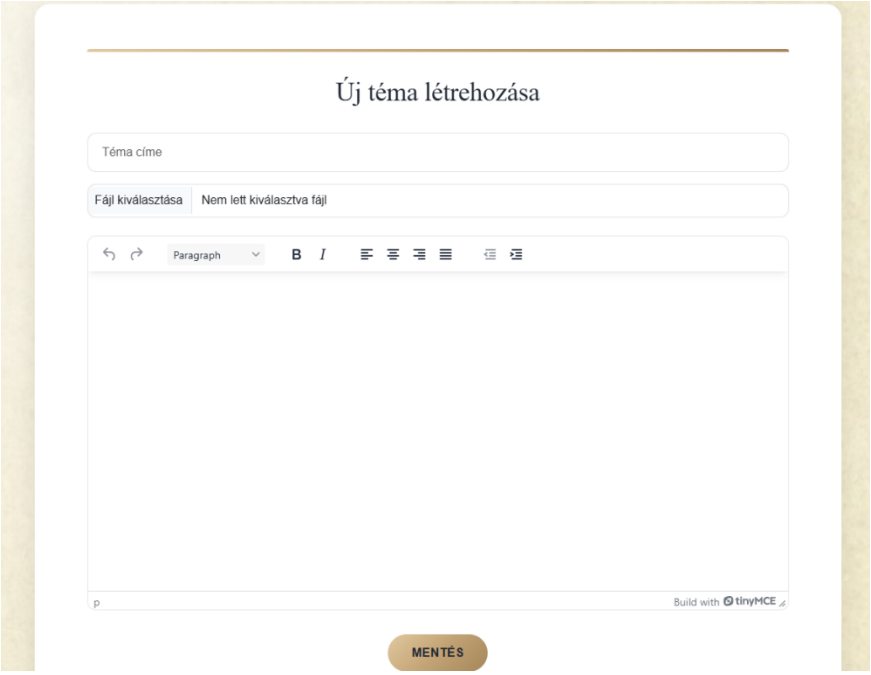
A Főoldal komponens az alkalmazás kezdőfelülete, amely a fő kategóriák megjelenítésével és keresési lehetőséggel segíti a felhasználói navigációt.

Főbb funkciók:

- Kategóriák kártyás megjelenítése
- Keresési funkció kulcsszavak alapján
- Navigáció kategória oldalakra
- Footer információk és linkek megjelenítése

Technológiai háttér:

- React hookok (useState, useNavigate)
- Bootstrap alapú reszponzív layout
- Dinamikus renderelés (map)



3. ábra: A CreateTopic komponens megjelenése a weboldalon

Layout komponens

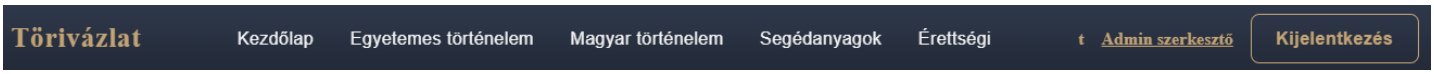
A Layout komponens az alkalmazás egységes keretrendszerét biztosítja, amely tartalmazza a navigációt és a dinamikus tartalom megjelenítését.

Főbb funkciók:

- Globális navigáció (Navbar)
- Felhasználói állapot megjelenítése
- Be- és kijelentkezés kezelése
- Admin felület elérésének biztosítása
- Oldalak közötti dinamikus tartalomkezelés (<Outlet />)

Technológiai háttér:

- React Router (Outlet, Link)
- Context API (AuthContext)
- Bootstrap komponensek



4.. ábra: A Layout komponens navbar elrendezése bejelentkezve adminisztrátorként

Login komponens

A Login komponens a felhasználók hitelesítéséért felel, lehetővé téve a rendszerbe történő bejelentkezést.

Főbb funkciók:

- Bejelentkezési űrlap kezelése
- Backend hitelesítés POST kérésen keresztül
- JWT token mentése sikeres bejelentkezéskor
- Hibakezelés és visszajelzés
- Navigáció regisztrációs oldalra

Technológiai háttér:

- React hookok és Context API
- REST API kommunikáció
- Bootstrap űrlap elemek

Main komponens

A Main komponens (belépési pont) a React alkalmazás inicializálásáért és az útvonalkezelés konfigurálásáért felel.

Főbb feladatok:

- Alkalmazás inicializálása (createRoot, StrictMode)
- Globális állapot biztosítása (AuthProvider)
- Útvonalak definiálása (React Router)
- Nyilvános és védett oldalak kezelése

Technológiai háttér:

- React 18+
- React Router DOM
- Globális állapotkezelés Context API-val

Register komponens

A Register komponens új felhasználók regisztrációját teszi lehetővé a rendszerben.

Főbb funkciók:

- Regisztrációs űrlap kezelése
- Kliensoldali validáció
- Adatok elküldése backend API-nak
- Sikeres regisztráció utáni átirányítás
- Hibakezelés

Technológiai háttér:

- React hookok (useState, useNavigate)
- REST API kommunikáció
- Bootstrap alapú felület

SearchResults komponens

A SearchResults komponens a keresési eredmények dinamikus megjelenítéséért felel.

Főbb funkciók:

- Kulcsszó alapú keresés
- Backend adatlekérés
- Találatok kártyás megjelenítése
- Navigáció a részletes oldalra
- Hibakezelés és „nincs találat” állapot

Technológiai háttér:

- React hookok (useEffect, useParams)
- REST API kommunikáció
- Dinamikus lista renderelés

TopicDetailPage komponens

A TopicDetailPage komponens egy adott történelmi téma részletes megjelenítését biztosítja.

Főbb funkciók:

- Téma adatainak lekérése

- HTML tartalom megjelenítése
- Admin funkciók (szerkesztés, törlés)
- Navigáció és visszalépés
- Betöltési és hibaállapot kezelése

Technológiai háttér:

- React hookok és routing
- REST API (GET, DELETE)
- JWT alapú hitelesítés

TopicEditor komponens

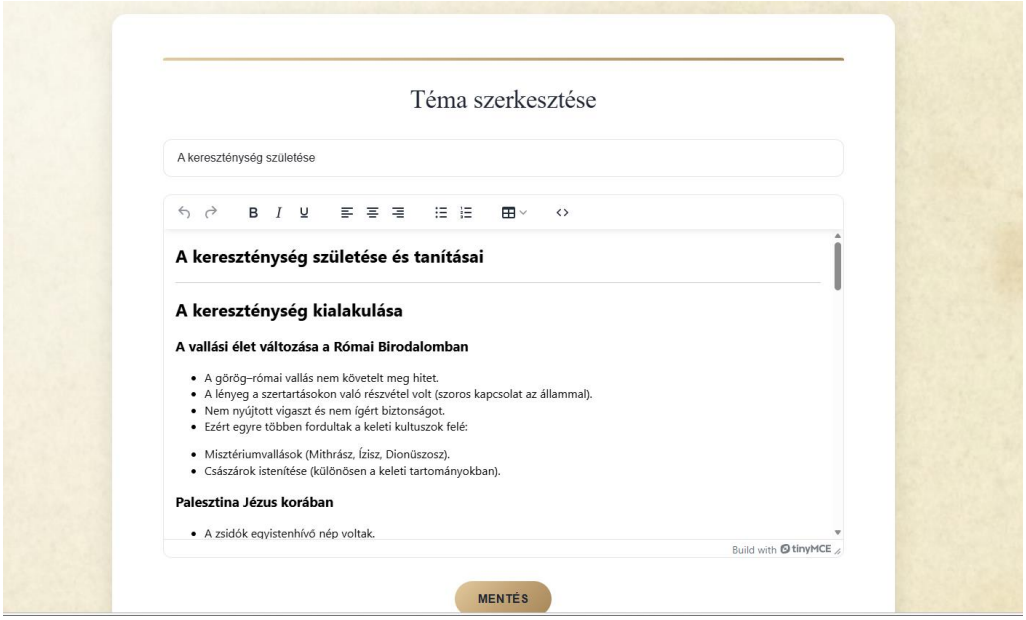
A TopicEditor komponens a meglévő tananyagok szerkesztésére szolgál adminisztrátori jogosultsággal.

Főbb funkciók:

- Téma adatainak betöltése
- Cím és tartalom szerkesztése
- TinyMCE editor használata
- Módosítások mentése backend felé
- Állapotkezelés (betöltés, mentés)

Technológiai háttér:

- React hookok
- TinyMCE integráció
- REST API (GET, PUT)



5. ábra: A TopicEditor komponens felülete a weboldalon

TopicPage komponens

A TopicPage komponens egy adott kategóriához tartozó témák listázásáért felelős (ld. 10. ábra).

Főbb funkciók:

- Témák lekérése kategória alapján
- Kártyás megjelenítés
- Navigáció a részletes oldalra
- Admin funkció (új téma létrehozása)
- Hibakezelés

Technológiai háttér:

- React hookok és routing
- REST API kommunikáció
- Dinamikus lista renderelés

UserEditor komponens

A UserEditor komponens az admin felületen a felhasználók kezelését biztosítja.

Főbb funkciók:

- Felhasználók listázása
- Jogosultság módosítása
- Felhasználók törlése
- Lista frissítése műveletek után
- Betöltési állapot kezelése

Technológiai háttér:

- React hookok (useState, useEffect)
- REST API (GET, PUT, DELETE)
- Bootstrap alapú megjelenítés

6. ábra: A UserEditor komponens szerkesztőfelülete a weboldalon

A webalkalmazás vizuális megjelenése:

Színvilág és vizuális identitás

A felület színrendszere tudatosan korlátozott, harmonikus és következetes:

- **Sötét alapszínek** (--primary-dark, --primary-dark-deep)

  Stabil és elegáns hátteret biztosítanak, amely kiemeli a tartalmat és fókuszálja a felhasználói figyelmet.

- **Arany árnyalatok** (--primary-gold, --gold-light, --gold-dark)

  A vizuális identitás meghatározó elemei, amelyek:

  - prémium hatást keltenek,
  - vizuális hierarchiát alakítanak ki,
  - kiemelik az interaktív elemeket.

- **Gradiens használat** (--gold-gradient)
  A finom színátmenetek dinamizmust adnak a felületnek anélkül, hogy túlzottan hangsúlyossá válnának.
- **Textúrázott háttér**
  A papírszerű háttér vizuálisan is erősíti a „digitális tankönyv” koncepciót, és kulturális kontextusba helyezi a tartalmat.

Tipográfia

A tipográfia a design egyik legfontosabb eleme, amely kontrasztra és jól definiált hierarchiára épül:

- **Címbetűtípus – Playfair Display**
  Klasszikus serif betűtípus, amely elegáns és művészeti jellegű megjelenést biztosít, elsősorban címsorokhoz használva.
- **Törzsszöveg – Inter**
  Modern, letisztult sans-serif betűtípus, amely kiváló olvashatóságot biztosít digitális környezetben.
- **Tipográfiai hierarchia**
  - jól elkülönülő címszintek (h1, h2),
  - kiegyensúlyozott betűközhasználat,
  - átlátható szövegszerkezet.
- **Speciális elemek**
  - iniciálék alkalmazása,
  - blokkidézetek kiemelése,
  - nagybetűs, ritkított címsorok.

A tipográfiai megoldások egyszerre utalnak a nyomtatott kiadványok világára és felelnek meg a modern UI design elvárásainak.

Layout és térközhasználat

Az elrendezés kialakítása a vizuális egyensúlyra és a megfelelő térhasználatra épül:

- nagy szekcióközök (--section-spacing),
- kártya alapú elrendezés,
- reszponzív grid rendszer (Bootstrap).

A whitespace tudatos alkalmazása:

- növeli az olvashatóságot,
- segíti a felhasználói fókuszt,
- prémium megjelenést biztosít.

Komponens alapú design

A felület moduláris, újrahasznosítható komponensekből épül fel:

Navigáció

- áttetsző háttér blur effekttel,
- lebegő vizuális hatás,
- finom hover animációk.

Hero szekció

- teljes szélességű háttérkép,
- sötét overlay a jobb olvashatóság érdekében,
- hangsúlyos tipográfia.

Kártyák

- lekerekített sarkok,
- árnyékok a mélység érzetéhez,
- hover effektek (emelkedés, nagyítás).

Gombok

- kapszula forma,
- arany gradiens háttér,
- finom interakciók.

Interakció és animáció

A felület dinamikáját visszafogott, de tudatosan alkalmazott animációk biztosítják:

- gyors és sima átmenetek (transitionök),
- hover effektek (kiemelés, zoom),
- belépési animációk (fade + elmozdulás).

Ezek az elemek:

- vizuális visszajelzést adnak,
- élőbbé teszik a felületet,
- javítják a felhasználói élményt.

Reszponzivitás

A webalkalmazás minden eszközön optimalizált megjelenítést biztosít:

- mobilon középre igazított tartalom,
- adaptív hero szekció,
- rugalmas grid rendszer,
- optimalizált képméretek.

Felhasználói élmény (UX)

A vizuális és interakciós megoldások célja egy intuitív és hatékony felhasználói élmény biztosítása.

A rendszer támogatja:

- az egyszerű navigációt,
- a gyors információfeldolgozást,
- az átlátható tartalomkezelést.

Ennek eredményeként a felhasználó könnyen eligazodik a felületen, és hatékonyan használhatja mind a tanulási, mind az adminisztrációs funkciókat.

Backend áttekintés:

Főbb funkciók

A backend több, egymással szorosan együttműködő modulból áll:

- **Felhasználókezelés**
  - regisztráció és bejelentkezés
  - jogosultsági szintek kezelése (admin / felhasználó)
  - felhasználók listázása, törlése, szerepkör módosítása
- **Tartalomkezelés**
  - témák létrehozása, módosítása és törlése
  - kategória alapú lekérdezések
  - részletes tartalomlekérés és keresési funkció
- **Képkezelés**
  - képfeltöltés Multer segítségével

    - fájlrendszerben történő tárolás
    - statikus kiszolgálás dedikált útvonalon
- **Hitelesítés és jogosultságkezelés**
    - jelszavak bcrypt alapú titkosítása
    - JWT token alapú autentikáció
    - admin jogosultság ellenőrzése védett végpontokon
- **API működés**
    - egységes HTTP státuszkódok használata
    - strukturált hibakezelés
    - JSON alapú kommunikáció

Alkalmazott technológiák

- **Node.js + Express** – szerveroldali logika és API kezelés
- **MySQL (mysql2)** – relációs adatbázis
- **Multer** – fájlfeltöltés kezelése
- **bcrypt** – jelszavak biztonságos hashelése
- **jsonwebtoken (JWT)** – token alapú hitelesítés
- **CORS** – frontend és backend közötti kommunikáció biztosítása

Biztonsági megoldások

A backend több szinten biztosítja az adatok és a rendszer védelmét:

- JWT alapú hitelesítés minden módosító végponton
- szerepkör alapú jogosultságellenőrzés (admin műveletek védelme)
- jelszavak bcrypt-tel történő titkosítása
- fájlfeltöltés korlátozása:
    - csak képfájlok engedélyezettek
    - maximális méret: 5 MB

Képek és tartalom kezelése

- A feltöltött képek a szerveren kerülnek tárolásra, és statikusan érhetők el a /Torivazlatkepek útvonalon.

- A témákhoz opcionálisan rendelhető kép, amely támogatja a vizuális tanulási élményt.
- Az admin felületen base64 formátumú képfeltöltés is elérhető, ami megkönnyíti a tartalomszerkesztést.

Keresés és kategorizálás

- Kulcsszavas keresés a témák címében (/api/search?q=...)
- Kategória alapú szűrés és listázás
- Részletes tartalomlekérés egyedi azonosítók alapján

Ez a struktúra gyors és hatékony navigációt biztosít a tananyagok között.

Fejlesztési és skálázhatósági szempontok

A rendszer kialakítása lehetővé teszi a későbbi bővítést:

- új funkciók integrálása (pl. kommentek, értékelések, statisztikák)
- router alapú modularizáció lehetősége
- token alapú autentikáció → mobil vagy SPA integráció támogatása

Végpontok

Összes felhasználó lekérése

Metódus	URL	Body	Válasz
GET	/api/allUsers	üres	JSON (felhasználók listája)

Összes téma lekérése

Metódus	**URL**	**Body**	**Válasz**
GET	/api/allCategory	üres	JSON (témák listája)

Témák lekérése kategória alapján

Metódus	**URL**	**Body**	**Válasz**

GET	/api/topicsByCategory/:id	üres	JSON (témák listája)

Egy konkrét téma tartalma

Metódus	URL	Body	Válasz
GET	/api/content/:catId/:tpcId	üres	JSON (title, content)

Keresés cím alapján

Metódus	URL	Body	Válasz
GET	/api/search?q=keresett_szöveg	üres	JSON (találatok listája)

Kép feltöltés

Metódus	URL	Body	Válasz
POST	/upload	form-data (file)	JSON (kép URL)

Új téma létrehozása

Metódus	URL	Body	Válasz
POST	/api/insert	JSON	JSON

JSON formátuma:

```
{
"title": "Téma címe",
"category_id": 1,
"content": "Tartalom",
"img": "base64 string (opcionális)"
}
```

Regisztráció

Metódus	**URL**	**Body**	**Válasz**
POST	/api/register	JSON	JSON

JSON formátuma:

{

"username": "felhasználó",

"email": "email@email.com",

"password": "jelszó"

}

Bejelentkezés

Metódus	URL	Body	Válasz
POST	/api/login	JSON	JSON (token)

JSON formátum:

{

"email": "email@email.com",

"password": "jelszó"

}

Téma módosítása (admin)

Metódus	URL	Body	Válasz
PUT	/api/topics/:catId/:tpcId	JSON + token	JSON

JSON formátum:

{

"title": "Új cím",

"content": "Új tartalom"

}

Téma törlése (admin)

Metódus	URL	Body	Válasz
DELETE	/api/topics/:catId/:tpcId	token	JSON

Felhasználó jogosultság módosítása (admin)

Metódus	URL	Body	Válasz
PUT	/api/users/:id/role	JSON + token	JSON

JSON formátum:

```
{
  "auth_id": 1
}
```

Felhasználó törlése (admin)

Metódus	URL	Body	Válasz
DELETE	/api/users/:id	token	JSON

Megjegyzés

- Az admin jogosultságot az auth_id = 1 jelöli
- A védett végpontok JWT token alapú hitelesítést használnak
- A token érvényességi ideje: **1 óra**

Projektértékelés és közreműködők

Projekt címe:

Történelem Vázlat (Törivázlat) projekt

Intézmény:

Siófoki SZC Mathiász János Technikum, Gimnázium és Kollégium

Szoftverfejlesztő és – tesztelő képzés

Szakmai konzulens és megrendelő:

Holczman József, történelem tanár

Fejlesztők:

- **Szabó Máté** – frontend fejlesztés (React), felhasználói felület (UI/UX) tervezés, fejlesztői dokumentáció
- **Zsupos Dominik** – adatbázis-tervezés (MySQL), backend fejlesztése (Node.js), felhasználói dokumentáció, tesztelési napló

Fejlesztési időszak:

2025. november 11. – 2026. április 20.

Használt technológiák és eszközök:

- HTML, CSS, JavaScript (frontend)
- Node.js (backend)
- MySQL (adatbázis)
- Trello (projektmenedzsment)
- Adobe Photoshop (koncepciótervezés)
- Git, GitHub (verziókezelés)
- Visual Studio Code (fejlesztői környezet)
- Postman, Thunder Client (API teszteléshez)

Tartalom

Rendszer architektúrája 2

Frontend (kliens oldal) 2

Backend (szerver oldal) 2

Adatbázis réteg 2

Adatáramlás folyamata 2

Funkciók bemutatása 3

Adatbázis 3

Alkalmazott táblák 3

Táblák közötti kapcsolatok 4

Kulcsok és adatintegritás 4

Adatbázis-struktúra 5

Frontend komponensek 5

AdminDashboard komponens 5

AdminRoute komponens 6

AllTopic komponens 7

AuthContext komponens 7

CreateTopic komponens 7

Főoldal komponens 8

Layout komponens 9

Login komponens 10

Main komponens 10

Register komponens 10

SearchResults komponens 11

TopicDetailPage komponens 11

TopicEditor komponens 12

TopicPage komponens 13

UserEditor komponens 13

A webalkalmazás vizuális megjelenése: 14

Színvilág és vizuális identitás 14

Tipográfia 15

Layout és térközhasználat 15

Komponens alapú design 16

Interakció és animáció 16

Reszponzivitás 17

Felhasználói élmény (UX) 17

Backend áttekintés: 17

Főbb funkciók 17

Alkalmazott technológiák 18

Biztonsági megoldások 18

Képek és tartalom kezelése 18

Keresés és kategorizálás 19

Fejlesztési és skálázhatósági szempontok 19

Végpontok 19

Projektértékelés és közreműködők 22

Projekt címe: 22

Intézmény: 22

Szakmai konzulens és megrendelő: 23

Fejlesztési időszak: 23

Használt technológiák és eszközök: 23